

```

    apiVersions: [ 1, Infinity ],
    serverVersions: [ '3.2.0', '999.999.999' ],
    returnsPromise: true,
    topologies: [ 'ReplSet', 'Sharded',
'LoadBalanced', 'Standalone' ],
    returnType: { type: 'unknown', attributes: {} },
    deprecated: false,
    platforms: [ 0, 1, 2 ],
    isDirectShellCommand: false,
    acceptsRawInput: false,
    shellCommandCompleter: undefined,
    help: [Function (anonymous)] Help
  }

```

You can run script with the shell as follows:

```

option 1> mongo script1.js script2.js
option 2> load("script1.js")
example>
    show dbs
    db.getMongo().getDBs()

```

MongoDB Compass is an interactive tool for querying, optimising, and analysing the MongoDB data. You can drag and drop it to build pipelines and more.

MongoDB is widely used across various web applications as the primary data store. One of the most popular web development stacks, the MEAN stack, employs MongoDB as the data store (MEAN stands for MongoDB, ExpressJS, AngularJS, and NodeJS). Other languages, such as Python, Java and Ruby, also have client libraries to work with MongoDB.

Importing and exporting data

The commands given below help to get and set the data from or to the MongoDB database.

JSON:

- **Import**
 - `mongodump --uri="mongodb+srv://<your username>:<your password>@<your cluster>.mongodb.net/sample_supplies"`
 - `--uri` (uniform resource identifier, srv establishes secure connection)
 - `srv` : connection string - a specific format used to establish a connection between your application and a MongoDB instance.
- **Export**
 - `mongoexport --uri="mongodb+srv://<your username>:<your password>@<your cluster>.mongodb.net/sample_supplies" --collection=sales --out=sales.json`

BSON:

- **Import**
 - `mongoimport --uri="mongodb+srv://<your username>:<your password>@<your cluster>.mongodb.net/sample_supplies" --drop sales.json`
- **Export**
 - `mongorestore --uri "mongodb+srv://<your username>:<your password>@<your cluster>.mongodb.net/sample_supplies" --drop dump`

Queries in MongoDB

In MongoDB, queries are commands or instructions used to retrieve data from a MongoDB database. MongoDB queries allow you to specify criteria to filter, sort, project, and aggregate documents within collections, which is critical when working with data.

- *findOne* -> Will return one document (row) from the collection
- *insertOne* -> One document will be inserted
- *updateOne* -> One document will be updated
- *deleteOne* -> One document will be deleted
- *find* -> Returned records are not ordered

```

db.zips.find({"state": "NY"})
    # results of it iterates through the cursor.
    # cursresultsor: pointer of the result set of query
    # pointer: a direct address of the memory location
db.zips.find({"state": "NY"}).count()
db.zips.find({"state": "NY", "city": "ALBANY"})
db.zips.find({"state": "NY", "city": "ALBANY"}).pretty()

```

updateOne and updateMany: These take a filter document as their first parameter. The second parameter is a modifier document that describes the changes to make.

replaceOne: These take a filter document as their first parameter, and the second parameter will replace the document matching the filter.

drop: When all collections are dropped from a database, the database no longer appears in the list of databases when you run it.

The preferred methods to avoid race conditions are:

- `findOneAndDelete`
 - `findOneAndUpdate`
 - `findOneAndReplace`
- Aggregation operations process multiple documents and return computed results.

Cursor methods

A cursor is a pointer to the result, from which we can access the